Exercises · RAG

AI05 Exercises

Work each one before reading its solution. Exercises 1, 3 and 5 include arithmetic. Practise Exercise 6 out loud.

Exercise 1: Do you need RAG?

A compliance team has a 900-page manual, at about 400 tokens per page. Everyone can read all of it. It's updated quarterly. They ask about 200 questions a day.

(a) How many tokens is it? **(b)** Does Anthropic's rule of thumb suggest stuffing? **(c)** Propose two designs. **(d)** What would decide between them?

Solution

(a) 900 × 400 = **360,000 tokens.**

(b) It's above the guidance of about 200,000 tokens, so the guidance doesn't settle it. But the guidance isn't a hard limit, and it fits in a 1M-token window.

(c) Design one: split by chapter, with a tool or router that loads the relevant chapters into a cached prompt. Design two: hybrid RAG with Contextual Retrieval.

(d) Run an eval on real questions. Compare answer quality, retrieval misses and cost at 200 questions a day, noting that the traffic may be too spread out to keep a 5-minute cache warm. A permission split would favor RAG, but here there's none.

Exercise 2: Audit a chunker

Your chunker produced these two chunks from a table:

```
Chunk 41: "Plan | Monthly | Annual
Basic | $10 | $100
Pro | $25"
Chunk 42: "| $250
Enterprise | Contact us | Contact us"
```

(a) List the problems. **(b)** Which user question fails? **(c)** Fix it. **(d)** Write a test that would have caught this.

Solution

(a) A row is split. Chunk 42 has no headers, so `$250` means nothing. There's also no heading saying what the table's about.

(b) "What does Pro cost annually?" Search may find chunk 41, which lacks the answer, or chunk 42, which lacks its meaning.

(c) Keep the table whole if it fits, and otherwise repeat the header on every piece. Add a heading path, such as `Pricing > Plans:`. Or store each row as a sentence: "Pro plan: $25 monthly, $250 annual."

(d) A fact-integrity test, as in `ChunkLab`: list facts such as "Pro | $25 | $250" and assert that one chunk contains each. Run it across sample documents in CI.

Exercise 3: Find the orphan chunks

A document version has 12 chunks in the index. An edit reduces it to 9. Your pipeline upserts by `document_id + chunk_no`.

(a) What's left behind? **(b)** Give a query to detect this. **(c)** Design the fix. **(d)** Suppose you have 2 million documents at 10 chunks each, using int8 and a graph at 1,288 bytes per chunk. If 1% of the chunks were orphans, how much memory is being wasted?

Solution

(a) Chunks 9, 10 and 11 of the old version (numbering from 0) remain searchable, and they might contradict the new text.

(b) Find indexed chunks whose `version` isn't the document's `active_version`. Before versioning exists, compare the index's chunk counts per document with the source's.

(c) Write the complete new version under a new version number, flip `active_version` in one transaction, and filter search by it. A sweeper then deletes old versions.

(d) 20 million chunks × 1,288 bytes = 25.76 GB in total, and 1% is about **0.26 GB.** The memory hardly matters. **Wrong answers are the real cost.**

Exercise 4: Diagnose an eval

Segment	Cases	Recall	Faithful
Policy lookups	80	72 of 80	70 of 80
Multi-document	50	45 of 50	25 of 50
Error codes	20	18 of 20	17 of 20

(a) Compute each rate, and overall rates. **(b)** What does the overall view hide? **(c)** Propose three fixes, in order. **(d)** How would you confirm a fix, given the segment sizes?

Solution

(a) Recall is 0.90 in every segment, and 135 of 150 = 0.90 overall. Faithfulness is 0.875, 0.50 and 0.85, and 112 of 150 = 0.75 overall.

(b) Retrieval's uniform, so the problem isn't retrieval. A 0.75 overall looks like general weakness, when really it's one segment at 0.50.

(c) All in generation for multi-document questions: (1) require a claim per source, with citations; (2) put documents first, with clear tags and dates; (3) extract relevant quotes first, then answer from them (AI03 L2).

(d) Re-run the same 50 cases, and check which ones changed, as ±14% at 50 cases makes independent comparisons weak. Have humans review a sample, then add more multi-document cases.

Exercise 5: Reduce the bill

A service gets 30,000 questions on each of 22 working days. Each uses 6,000 input tokens of context, and 500 output tokens, on Sonnet 5.5 ($2 / $10 per million).

(a) What's the monthly cost? **(b)** A rerank change halves the context to 3,000 tokens. What's the new cost? **(c)** What must you show before shipping it? **(d)** Why might this change also improve quality?

Solution

(a) 660,000 questions × ($0.012 + $0.005) = **$11,220.**

(b) 660,000 × ($0.006 + $0.005) = **$7,260**, saving $3,960 a month.

(c) That recall at the new, smaller cut doesn't fall in any segment, nor does faithfulness or correctness. Multi-document questions are the likeliest to suffer.

(d) Fewer irrelevant chunks can reduce distraction. This is plausible, not guaranteed, so measure it.

Exercise 6: Defend a decision

A VP says: "We've chosen a graph database and an agent for our knowledge assistant. Everyone's moving to GraphRAG and agents. Vector search is dead."

The company's questions are mostly policy lookups. It has 200,000 documents and strict permissions.

Respond in about a minute. Cover evidence, risk, a counter-proposal, and what would change your mind.

Model answer

"I'd like us to decide this from our questions. Let's sample a few hundred from tickets and search logs. I expect most to be policy lookups, which is where hybrid search works well. Vector search is one half of that, along with keyword search, and it isn't dead.

Graph RAG exists for whole-corpus and multi-hop questions, as its authors describe. It runs every document through an LLM before anyone asks anything, and extraction errors become authoritative-looking edges. Agents cost more and vary more per question, and that's a poor fit for a simple lookup. Permissions are our highest risk, and each new retrieval path becomes something else to secure.

My proposal: start with hybrid search, permission filters and citations, with evaluation by question type. In parallel, add a router so that specific question types can get their own route later.

I'd change my mind if evaluation showed many questions failing because they span documents, or concern the whole corpus. Then we'd add a graph or a batch summary for those questions, with evidence that the added cost pays for itself."